pyAmpliCol at one loop

Born–virtual interference: milestone status, benchmarks and architecture

Milestone 2026-10-24 · engine revision b4d76cc3 · AMD EPYC 9754 (shared host)

pyAmpliCol computes the interference $2\,\mathrm{Re}\left(B^{\dagger}GL\right)$ of one-loop and Born amplitudes with the UV and R_2 counterterms of the UFO loop model, for every squared coupling order and all three Laurent coefficients, in binary64, double-double (DD) and arbitrary precision (AP). The engine is the recursive *v2* engine: tree currents, loop topologies and loop numerators come from recursions, and no loop Feynman diagram is enumerated (Section 5). Counterterms come from the tree recursion (WP2.2) as plain tree vertices inserted exactly once, admitted by coupling orders and a public `particle_veto`, without loop matching (addendum J, on `loop` since b4d76cc3). This report records **gate G1**, the first end-to-end measurement of v2 against MadLoop, Recola and OpenLoops (Section 1), measured on the engine of the first 24 October push (59badc7a) with the native-DAG merge (b71cf0e0, no production call site switched) and, as a development configuration, on the WP3.1 branch whose native per-point inputs are not yet integrated.

Published on `loop` on 24 October (engine b4d76cc3; merges since f132e53c)

`b71cf0e0`	Rust A1-A2	**Native loop-request DAG builder.** Rust builder for the loop-block DAG and blocks, equal to the Python builder (whole GenericDAG and Block records, flat-record digests at seven legs); no production call site switched yet
`6c44c9f7`	WP2.7	**Gate G1 harness and measurement.** tools/loop_benchmark (specs, frozen streams, providers, median gate statistic with bands), cost model, this report's G1 section
`b4d76cc3`	addendum J	**Counterterms as plain tree insertions.** no witness table or loop matching in production; public particle_veto; an unlowerable counterterm vertex that no tree of the request reaches is inert and recorded

Before (f132e53c): v2 waves 0-1, gates S1 and G0, native executor (WP2.1), colour (WP2.4), stability API (WP2.3), schedule M (WP2.8), tree-EW follow-up, NJet reference, observables (WP2.5), public v2 engine with plan v4 artifacts (WP2.6), counterterms in the recursion (WP2.2), runtime pole selection (WP2.10), Rust A0 (contract-class reuse pass), constant counterterm arguments (ct-assume-fix), addendum F-J, the previous report. **Native build** `loop-native-v2-slice-6c44c9f7` (digest `a9533f0e`…).

Performance gates: policy and status

- **Gate.** Faster than MadLoop and at most 1.30× each matched Recola and OpenLoops time, on the **finite part**, for processes from $2 \to 3$ upward. $2 \to 2$ is reported without a gate.
- **Protocol.** Providers in production mode with poles off where they allow it (OpenLoops `truepoles 0`, Recola without IR poles, MadLoop production build); pyAmpliCol finite-only (`poles = "none"`) with stability checks off, one pinned thread, the same frozen points.
- **Status (Table 1, Table 2).** Integrated: fails every gated process it generates (×11–27; Python inputs), refuses V +jets. Native inputs^{dev} at $B = 128$: 7 of 9 pass, 1 open, 1 fails; at $B = 1$: 0 (16 lanes), 6 (one lane).
- **Seven legs** follow with WP3.1 (`measure-g1-7leg`); $gg \to ggggg$ without a gate (no matched provider, Table 10).
- **History.** The timings of 2026-10-18 measure the superseded per-diagram engine and appear only in Section 6.

G1 at $B = 128$: integrated fails 7 of 7 (2 not generable) · native inputs^{dev}: 7 pass, 1 open, 1 fail of 9

Paused 24 October, about 03:30 UTC (agent quota until 27 October; resume from `HANDOFF_24_10_2026.md`). On branches, not on `loop`:

- **Generation at scale** (WP3.1 G-int, `work/v2-generation-scale`): native trees, Born and counterterm inputs, regeneration-free artifacts; timed here as the native-input configuration. Not yet audited: after the slice merge its $gg \to ggggg$ (recursion counterterms, native inputs) still matches NJet (finite $\leq 4 \times 10^{-9}$ at three points); generation took 46 min against 5.4 min before the merge: about 13 min for the recursion counterterm program and 27 min for poles-uvir's always-generated companion (its seed table builds generic seeds; making it lean is open). Development measurements on a shared host.
- **Rust ports** (HANDOFF §0.4): A0 and A1–A2 are on `loop`, not yet switched on; seeds (B) and cells (C) passed their phase-0 audits and wait for G-int; lowering (D) is in an audit fix round.
- **Not started or partial:** `artifact-portability` (partial branch), `measure-g1-7leg`; **electroweak** and complex mass (WP3.2).

User decisions of 23 October

- **Counterterms** (addendum J): model R_2 and UV vertices are plain tree vertices inserted exactly once, admitted by coupling orders and a public `particle_veto`; no witness table or loop matching in production (site closure and witnesses: test oracles only).
- **Poles:** `"uv"` = UV poles of the bare loop, the counterterms' UV poles separate; `"ir"` = Catani poles of the renormalised virtual; one finite part.
- **Rust** (HANDOFF §0.4): heavy, contained generation stages move to rusticol (PyO3 API) with oracle equality; new heavy code in Rust.
- Generation may take O(hours) below 150 GB (addendum H); $gg \to ggggg$ speed without a gate, with rotation and Ward checks; no $2 \to 6$ before the $2 \to 5$ plan is complete.
- Binary64 observables: 10^{-12} unless the own estimate $E > 10^{-12}$ (then E, capped at 10^{-9}, plus DD within 10^{-24}).
- Stability-estimate tests: tenfold margin (error $\lesssim 10\,E$). Development first, proportionate testing; push to `loop` at milestones with this report.

1 Gate G1: end-to-end timings of the v2 engine

G1 times the complete Born–virtual evaluation, **finite part only**, through the call of the public v2 runtime (`NativeExecutor.evaluate`, which `V2VirtualRuntime` calls) with stability checks off, on one pinned core and one thread, binary64, 16 SIMD lanes, in batches of B distinct points of a frozen stream (REQUIREMENTS_ADDENDUM D, plan §5.8). Two pyAmpliCol configurations are timed:

- **integrated** — this report's branch at `1c971289` (LC rows at `d591a2f7`, the same engine code): the engine on `loop` since 24 October plus the native-DAG merge (no production call site switched) and this package's timers. Trees, Born amplitudes and counterterm scalars are formed per point in Python, then the loop stage runs natively. It refuses the V+jets rows: their closed-quark Agg triangles need Furry's certified zeros (WP3.5), and their Z couplings on loop quark lines the electroweak fermion signs (WP3.2);
- **native inputs** development — `work/v2-generation-scale 889af02c` (WP3.1, under integration; scratch commit `39824f9b` adds the timers), where the executor also forms the trees, the Born and the counterterm inputs. It generates the V+jets rows with γ and Z exchange.

The providers run in production mode on the same stream: OpenLoops with `truepoles 0`, Recola without IR poles (one amplitude call per point), MadLoop at its production stability setting on exports built with MG5′s production flags. OpenLoops has no installed library with QCD corrections to $\ell^+\ell^-$ + 1 or 2 jets (`pplj_ew` carries electroweak corrections only; existing provider builds only), so the V+jets gates bind on MadLoop and Recola. Providers have no batch API: one time per point serves every B. **Gate statistic:** the median per point on both sides — over pyAmpliCol's timed blocks of one B, and over the provider's timed points (the stream's first 16; 8 for MadLoop $gg \to gggg$, Recola $gg \to gggg$, OpenLoops $gg \to gggg$ and Recola $u\bar{u} \to gggg$) in its run on the core that timed the native-input configuration (other cores: MadLoop $gg \to gggg$, Recola $gg \to gggg$ and Recola $u\bar{u} \to gggg$). Its uncertainty is pyAmpliCol's block spread (half the range of three blocks, the median's standard error from five) plus the provider's half-range over its repeated runs (8 %, the campaign's largest, for the single runs of Recola $gg \to gggg$ and Recola $u\bar{u} \to gggg$). A gate passes or fails only if the whole band does; otherwise it is **open** and the configuration is repeated once (plan §5.8 item 10). Every timed call returned all its points (failed points: 0); rescue is off by protocol (no checks, no escalation), so the rescue rate is 0.

Table 1: **Gate G1, full colour**, ms per point (gate statistic: median), finite part, one thread. Rows $B = 1$ and $B = 128$ (the integrated $2 \to 4$ rows only at $B = 1$: their Python inputs take seconds per point). Multiplier and colour: time ÷ the fastest provider of the row (legend below). Grey rows: generation — pyAmpliCol generation plus plan export with its peak memory; MadLoop graph generation, export and compilation · the share of its loop-numerator coefficients in its time (Table 3); Recola process generation; OpenLoops: the prebuilt library and its load plus first call. MadLoop $u\bar{u} \to gggg$ was not exported: the $gg \to gggg$ export took 2.4 h here (HANDOFF §0.1), and OpenLoops binds the gate. Gate: upper line integrated, lower line native inputs; the number is the time over the binding limit (MadLoop's time or 1.30× the faster of Recola and OpenLoops); **open**: the uncertainty band contains the limit (Table 2 gives the bands).

Process	B	Integrated	Native inputs[dev]	MadLoop	Recola	OpenLoops	Gate
$gg \to gg$ $2 \to 2$, no gate	1	140 ×69	14.7 ×7.3	2.02 fastest	10.0 ×5.0	3.20 ×1.6	no gate
	128	132 ×66	2.22 ×1.1	2.02 fastest	10.0 ×5.0	3.20 ×1.6	no gate
	gen.	10.4 s · 0.3 GiB	9.82 s · 0.3 GiB	9.70 s · 64 %	0.142 s	ppjj · 6.80 ms	
$u\bar{u} \to d\bar{d}g$ $2 \to 3$	1	40.6 ×27	10.9 ×7.2	3.17 ×2.1	3.18 ×2.1	1.52 fastest	fail ×21 fail ×5.5
	128	32.4 ×21	2.65 ×1.8	3.17 ×2.1	3.18 ×2.1	1.52 fastest	fail ×16 fail ×1.3
	gen.	9.94 s · 0.3 GiB	10.6 s · 0.3 GiB	9.87 s · 73 %	0.406 s	ppjjj · 4.73 ms	
$u\bar{u} \to u\bar{u}g$ $2 \to 3$	1	68.9 ×24	19.1 ×6.5	8.59 ×2.9	5.55 ×1.9	2.92 fastest	fail ×18 fail ×5.0
	128	54.5 ×19	3.80 ×1.3	8.59 ×2.9	5.55 ×1.9	2.92 fastest	fail ×14 open 1.00
	gen.	10.4 s · 0.3 GiB	10.9 s · 0.3 GiB	10.9 s · 75 %	0.462 s	ppjjj · 8.43 ms	
$u\bar{u} \to e^-e^+g$ $2 \to 3$	1	not generable	3.62 ×3.8	0.948 fastest	1.08 ×1.1	no library	— fail ×3.8
	128	not generable	0.772 ×0.82	0.948 fastest	1.08 ×1.1	no library	— pass 0.82
	gen.	CT coverage refused	26.0 s · 0.4 GiB	14.0 s · 79 %	0.701 s	—	
$u\bar{u} \to ggg$ $2 \to 3$	1	215 ×17	50.6 ×4.1	28.2 ×2.3	25.3 ×2.0	12.4 fastest	fail ×13 fail ×3.1
	128	179 ×14	11.2 ×0.90	28.2 ×2.3	25.3 ×2.0	12.4 fastest	fail ×11 pass 0.69
	gen.	12.3 s · 0.4 GiB	12.2 s · 0.3 GiB	17.6 s · 76 %	0.634 s	ppjjj · 23.7 ms	
$gg \to ggg$ $2 \to 3$	1	2 760 ×28	211 ×2.2	247 ×2.5	351 ×3.6	97.0 fastest	fail ×22 fail ×1.7
	128	2 560 ×26	45.8 ×0.47	247 ×2.5	351 ×3.6	97.0 fastest	fail ×20 pass 0.36
	gen.	27.9 s · 0.6 GiB	14.8 s · 0.4 GiB	47.0 s · 78 %	6.63 s	ppjjj · 159 ms	
	1	not generable	43.3 ×4.2	27.3 ×2.7	10.2 fastest	no library	— fail ×3.3

Process	B	Integrated	Native inputs^{dev}	MadLoop	Recola	OpenLoops	Gate
$u\bar{u} \to e^-e^+gg$ $2 \to 4$	128	not generable	5.64 ×0.55	27.3 ×2.7	10.2 fastest	no library	— pass 0.42
	gen.	CT coverage refused	28.1 s · 0.4 GiB	24.8 s · 83 %	0.833 s	—	
$gg \to gggg$ $2 \to 4$	1	101 000 ×36	4 100 ×1.4	38 700 ×14	17 600 ×6.2	2 840 fastest	fail ×27 fail ×1.1
	128	not timed	985 ×0.35	38 700 ×14	17 600 ×6.2	2 840 fastest	— pass 0.27
	gen.	335 s · 4.1 GiB	96.1 s · 2.0 GiB	8 680 s · 51 %	2 130 s	ppjjjj · 4 950 ms	
$gg \to t\bar{t}gg$ $2 \to 4$	1	7 310 ×15	840 ×1.8	5 450 ×12	1 970 ×4.2	472 fastest	fail ×12 fail ×1.4
	128	not timed	190 ×0.40	5 450 ×12	1 970 ×4.2	472 fastest	— pass 0.31
	gen.	62.5 s · 0.9 GiB	36.6 s · 0.7 GiB	218 s · 64 %	61.4 s	ppttjj · 821 ms	
$u\bar{u} \to gggg$ $2 \to 4$	1	5 760 ×15	860 ×2.3	not timed	908 ×2.4	374 fastest	fail ×12 fail ×1.8
	128	not timed	190 ×0.51	not timed	908 ×2.4	374 fastest	— pass 0.39
	gen.	56.1 s · 0.7 GiB	39.4 s · 0.7 GiB	—	32.8 s	ppjjjj · 606 ms	

×1/16 faster ×1/4 ×1 ×4 time ÷ fastest reference, log scale, clamped ×16 slower

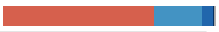
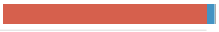
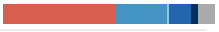
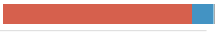
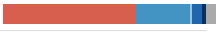
Table 2: **The gate at every batch size** (native inputs^{dev}, full colour): time over the binding limit and its uncertainty band (cell colour: blue pass, red fail, amber open). $B = 1$ is timed with the default 16 lanes (a one-point call pays a 16-lane chunk) and with one lane. The $u\bar{u} \to u\bar{u}g$ and $u\bar{u} \to d\bar{d}g$ rows are the single repeat; their first runs gave $u\bar{u} \to u\bar{u}g$ 3.80 ms and $u\bar{u} \to d\bar{d}g$ 2.63 ms at $B = 128$.

Process	$B = 1$, 16 lanes		$B = 1$, 1 lane		$B = 8$		$B = 32$		$B = 128$	
$u\bar{u} \to d\bar{d}g$	5.54	5.46–5.62	2.13	2.10–2.16	1.46	1.37–1.55	1.15	1.12–1.17	1.35	1.19–1.51
$u\bar{u} \to u\bar{u}g$	5.03	4.93–5.14	1.63	1.59–1.67	1.15	1.07–1.22	0.87	0.85–0.90	1.00	0.90–1.10
$u\bar{u} \to e^-e^+g$	3.82	3.81–3.83	1.61	1.60–1.62	1.07	1.01–1.13	0.85	0.83–0.87	0.82	0.81–0.82
$u\bar{u} \to ggg$	3.14	3.06–3.22	0.84	0.81–0.86	0.68	0.66–0.71	0.74	0.61–0.87	0.69	0.64–0.75
$gg \to ggg$	1.67	1.67–1.67	0.45	0.45–0.45	0.42	0.40–0.44	0.34	0.33–0.35	0.36	0.36–0.36
$u\bar{u} \to e^-e^+gg$	3.26	3.24–3.28	0.78	0.78–0.78	0.62	0.21–1.02	0.42	0.41–0.42	0.42	0.40–0.45
$gg \to gggg$	1.11	1.10–1.12	0.31	0.30–0.31	0.30	0.30–0.31	0.26	0.24–0.29	0.27	0.25–0.28
$gg \to t\bar{t}gg$	1.37	1.35–1.38	0.37	0.36–0.37	0.35	0.33–0.37	0.28	0.27–0.29	0.31	0.30–0.32
$u\bar{u} \to gggg$	1.77	1.74–1.80	0.43	0.43–0.44	0.47	0.43–0.51	0.37	0.32–0.41	0.39	0.37–0.41

1.1 Where the time goes

Table 3: **Stage split**, ms per point, full colour, at the largest batch timed (integrated $2 \to 4$: $B = 1$). Trees/Born/CT: the per-point inputs — in Python for the integrated slice (trees, counterterm program, Born and counterterm colour projections, scalars), in the executor with native inputs. Numerator: seeds, fused factors and the dressing of the open-loop objects (the pure numerator evaluation). Scatter: closures into NinjaRS lanes. NinjaRS: the reduction without the masters; masters: the master integrals (canonical arguments, cache, oneloopmaster). Glue: point parameters, carrier conversion, assembly and the rest of the Python call. MadLoop (instrumented copy; its points less one, 3 for $gg \to gggg$): Born/counterterm HELAS calls, loop-numerator coefficients, reduction plus masters, residual (stability test, colour sums). Bar: shares, colours as the column heads.

Process		Trees, Born, CT	Numerator	Scatter	NinjaRS	Masters	Glue	Total	Shares
$gg \to gg$	native^{dev}	0.941	0.836	0.0886	0.114	0.0378	0.206	2.22	
	integrated	128	0.851	0.0919	0.119	0.0381	2.96	132	
	MadLoop	0.0315	1.27	—	0.585		0.107	1.99	
$u\bar{u} \to d\bar{d}g$	native^{dev}	0.405	0.753	0.464	0.139	0.112	0.785	2.65	
	integrated	28.1	0.749	0.462	0.137	0.111	2.66	32.4	
	MadLoop	0.0479	2.35	—	0.683		0.133	3.22	
$u\bar{u} \to u\bar{u}g$	native^{dev}	0.768	1.22	0.472	0.223	0.161	0.970	3.80	
	integrated	48.5	1.23	0.479	0.232	0.171	3.99	54.5	
	MadLoop	0.137	6.44	—	1.15		0.814	8.54	
$u\bar{u} \to e^-e^+g$	native^{dev}	0.128	0.266	0.151	0.0266	0.0179	0.182	0.772	
	MadLoop	0.0184	0.754	—	0.133		0.0460	0.951	
$u\bar{u} \to ggg$	native^{dev}	5.85	2.94	0.480	0.468	0.228	0.535	11.2	
	integrated	168	2.92	0.476	0.459	0.225	5.75	179	
	MadLoop	0.346	18.5	—	2.54		2.87	24.3	
$gg \to ggg$	native^{dev}	28.3	11.8	0.538	2.05	0.487	2.76	45.8	
	integrated	2 530	11.7	0.519	2.03	0.483	10.5	2 560	
	MadLoop	2.42	184	—	11.3		39.5	237	
$u\bar{u} \to e^-e^+gg$	native^{dev}	1.13	2.75	0.637	0.423	0.233	0.459	5.64	
	MadLoop	0.289	22.7	—	2.38		2.11	27.5	

Process		Trees, Born, CT	Numerator	Scatter	NinjaRS	Masters	Glue	Total	Shares
$gg \to gggg$	native^{dev}	696	221	2.42	47.9	5.85	5.35	985	
	integrated	97 300	3 750	4.08	38.6	5.81	137	101 000	
	MadLoop	88.9	18 600	—	209		17 700	36 500	
$gg \to t\bar{t}gg$	native^{dev}	102	45.9	0.923	20.4	6.31	15.0	190	
	integrated	6 490	751	3.55	20.7	6.58	29.3	7 310	
	MadLoop	24.2	3 290	—	105		1 740	5 160	
$u\bar{u} \to gggg$	native^{dev}	119	47.9	1.81	9.14	2.90	8.49	190	
	integrated	4 910	806	2.01	9.64	2.99	24.1	5 760	

Verdict of gate G1 (processes from $2 \to 3$; $2 \to 2$ without a gate).

- **Integrated slice: fails** every process it generates, at ×11–27 of the binding limit at the largest batch and ×12–27 at $B = 1$; the per-point Python inputs take 85 %–99 % of its time (Table 3). It does not generate $u\bar{u} \to e^-e^+g$ and $u\bar{u} \to e^-e^+gg$ (counterterm coverage: Furry zeros, WP3.5; Z fermion signs, WP3.2).
- **Native inputs** development at $B = 128$: **pass** for $u\bar{u} \to e^-e^+g$, $u\bar{u} \to ggg$, $gg \to ggg$, $u\bar{u} \to e^-e^+gg$, $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$ **fail** for $u\bar{u} \to d\bar{d}g$ (×1.3 of the limit, OpenLoops binding); **open** for $u\bar{u} \to u\bar{u}g$ (1.00 of its limit after the single repeat; 0.87 at $B = 32$). The gate holds for every B (Table 2): at $B = 1$ with 16 lanes none passes (a one-point call pays a 16-lane chunk); with one lane per call $u\bar{u} \to ggg$, $gg \to ggg$, $u\bar{u} \to e^-e^+gg$, $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$ pass.
- **Fastest of all tools** (aspiration), native inputs at $B = 128$, within the uncertainty band: faster than every provider for $u\bar{u} \to e^-e^+g$, $u\bar{u} \to ggg$, $gg \to ggg$, $u\bar{u} \to e^-e^+gg$, $gg \to gggg$, $gg \to t\bar{t}gg$ and $u\bar{u} \to gggg$; at parity with the fastest for none; slower than the fastest (OpenLoops) for $u\bar{u} \to d\bar{d}g$ and $u\bar{u} \to u\bar{u}g$. NJet, not a matched provider, takes 4.58 s for its two-evaluation full-colour $gg \to gggg$ test (Table 13).
- Seven-leg rows ($u\bar{u} \to e^-e^+d\bar{d}g$, $u\bar{u} \to e^-e^+ggg$, $gg \to t\bar{t}e^-e^+g$, $u\bar{u} \to ggggg$, $gg \to ggggg$) need the WP3.1 generator and are added by the follow-on package `measure-g1-7leg`; $gg \to ggggg$ has no matched provider timing (addendum F, H): reported without a gate.

Table 4: **Levers**, measured on the rows of Table 3. Gains are measured where a column says so; otherwise they are the stage's share, an upper bound.

Lever	Evidence	Expected gain	Owner
Native per-point inputs (WP3.1 I3–I6: trees, Born, counterterm scalars in the executor)	Python inputs 85 %–99 % of the integrated time	measured, integrated ÷ native at equal B: ×3.6–25 at $B = 1$, ×12–56 at $B = 128$ ($2 \to 3$)	`generation-scale` (G-int, branch; audit pending)
Compiled native trees (kernels as the tree engine's SymJIT planes, lanes batched)	native trees 15 %–71 % of the native-input time; $gg \to gggg$ 71 %	up to the share; ×3.4 for the largest	Rust ports A/B (`native-dag`, HANDOFF §0.4)
Lanes by batch in binary64 (`plan_export.lanes_for`: 16 lanes also for one point)	$B = 1$: one 16-lane chunk per call	measured: ×2.4–4.2 at $B = 1$ (16 lanes ÷ 1 lane)	executor policy (WP3.1 or wave 4)
Finite-only result path (sum the virtual natively, no per-family lane export, Python assembly only for checks and observables)	glue 1 %–30 % of the native-input time	up to the share (largest at $2 \to 3$ with quarks)	`plan_export`, executor (wave 4)
Lane-major closures (no scatter, no second copy into the reducer's batch)	scatter 0 %–20 %	up to the share	executor, reducer adapter (wave 4)
AVX2 / AVX-512 kernels (runtime selection)	numerator 22 %–49 %	×1.0–1.4 on the dressing (gate G0)	wave 4 (WP4.6)
NinjaRS and masters: parallel reduction, transposed reduction, stage-2 sharing, a cheaper master cache key	NinjaRS + masters 5 %–14 %; masters 11 %–45 % of the reduction, oneloopmaster alone 2 %–12 %: the cache bookkeeping costs more than the integrals	one thread: up to the share; threads: DESIGN §5.6	NinjaRS requests (DESIGN §11.5)

The failing native-input row, $u\bar{u} \to d\bar{d}g$, takes 2.65 ms at $B = 128$ against a limit of 1.97 ms (OpenLoops 1.52 ms × 1.30). Its glue and scatter are 1.25 ms; without them it would take 1.40 ms, 0.71 of the limit. At $B = 32$ it takes 2.26 ms: its glue grows with the batch (0.325 ms per point at $B = 32$, 0.785 ms at $B = 128$).

1.2 Leading colour

Table 5: **Leading colour** (LC), ms per point, same protocol as Table 1; in each cell the full-colour (FC) time of the same configuration and LC/FC. No provider ratio: the providers time full colour. LC uses the committed QCD colour continuation (`examples/loop/source_colour`); the V+jets rows were not timed at LC.

Process	B	Integrated		Native inputs[dev]	
$u\bar{u} \to d\bar{d}g$	1	40.5	FC 40.6 · 1.00	5.25	FC 10.9 · 0.48
	128	37.4	FC 32.4 · 1.15	1.91	FC 2.65 · 0.72
$u\bar{u} \to u\bar{u}g$	1	not timed		9.00	FC 19.1 · 0.47
	128	not timed		3.16	FC 3.80 · 0.83
$u\bar{u} \to ggg$	1	184	FC 215 · 0.86	22.8	FC 50.6 · 0.45
	128	171	FC 179 · 0.96	12.6	FC 11.2 · 1.13
$gg \to ggg$	1	2 350	FC 2 760 · 0.85	91.1	FC 211 · 0.43
	128	not timed		50.7	FC 45.8 · 1.11
$gg \to gggg$	1	not timed		2 070	FC 4 100 · 0.50
	128	not timed		1 090	FC 985 · 1.10
$gg \to t\bar{t}gg$	1	not timed		418	FC 840 · 0.50
	128	not timed		231	FC 190 · 1.21
$u\bar{u} \to gggg$	1	not timed		380	FC 860 · 0.44
	128	not timed		247	FC 190 · 1.30

LC is cheaper in the loop stage (fewer families and skeletons), but not yet end to end: with native inputs the trees at LC cost more than at full colour for the processes with many colour orderings ($u\bar{u} \to ggg$: 9.93 against 5.85 ms per point), and in the integrated slice the Python inputs dominate either way.

1.3 Throughput, model recalibration and schedule M

Table 6: **Kernel throughput and the design recipe** (native inputs, full colour, largest batch). Work: the executor's modelled complex multiply-adds of the chosen kernel forms per point (one lane); throughput: work × lanes over the dressing time (fused-factor builds excluded). Native loop: seeds, factors, dressing, scatter and reduction; outside: everything else of the call (trees, glue). Recipe: DESIGN §1.3, finite only, where the design has a row; measured ÷ recipe for the total and for the NinjaRS stage (masters included) against its model R.

Process	Work, MCMAC/pt	Dressing, ms	G CMAC/s	Native loop, ms	Outside, ms	Total, ms	Total ÷ recipe	NinjaRS ÷ R
$gg \to gg$	0.932	0.694	1.34	1.08	1.14	2.22	1.5–4.4	0.97–1.10
$u\bar{u} \to d\bar{d}g$	0.556	0.446	1.25	1.47	1.19	2.65	no row	—
$u\bar{u} \to u\bar{u}g$	1.09	0.864	1.26	2.08	1.73	3.80	no row	—
$u\bar{u} \to e^-e^+g$	0.168	0.131	1.28	0.462	0.311	0.772	no row	—
$u\bar{u} \to ggg$	3.18	2.44	1.30	4.12	7.07	11.2	no row	—
$gg \to ggg$	14.3	10.9	1.32	14.9	30.9	45.8	no row	—
$u\bar{u} \to e^-e^+gg$	2.89	2.22	1.30	4.05	1.59	5.64	no row	—
$gg \to gggg$	268	215	1.25	278	708	985	2.4–7.1	1.05–1.24
$gg \to t\bar{t}gg$	56.0	43.2	1.29	73.5	117	190	1.2–3.5	0.99–1.39
$u\bar{u} \to gggg$	56.7	43.6	1.30	61.8	128	190	no row	—

Recalibration. The kernels run 1.25–1.34 G complex MAC/s per core on the real plans, the rate gate G0 measured on its synthetic programs (1.29–1.37); the dressing term of the recipe holds. The NinjaRS stage, masters included, lands on the design's model R (measured ÷ R 0.97–1.39, Table 6). What the recipe misses is the **outer factor**: it assumed trees, seeds and counterterms add 5–10 % (DESIGN §1.3), but with the current native inputs the total is 1.66–3.6 times dressing plus reduction at full colour (up to 13 at LC). Table 7 re-estimates every design row with these measured terms.

Table 7: **Design estimates recalibrated by G1** (`tools/loop_v2_costmodel.py --g1`), ms per point, finite only, one thread. Recipe: DESIGN Table 1a's finite-only band (outer factor 1.05–1.10). G1: the recipe's dressing term, the NinjaRS model R × 0.97–1.39 (measured above) and the measured outer factor of the row's class representative at $B = 128$ with native inputs (same colour where measured; LC rows of the V+jets class take the FC factor, marked †). Measured: G1's own time where the row was timed. Gate: 1.3× the best production provider of DESIGN Table 1b (seven-leg rows, protocol of 20 October), with the headroom gate ÷ upper estimate. The outer factor shrinks with compiled trees (Table 4); until then these replace the Table 1a targets.

Design row	Col.	Recipe	Outer	Class	G1 estimate	Measured	Gate	Headroom
$gg \to gg$	FC	0.502–1.52	2.25	$gg \to gg$	1.07–3.25	2.22	—	—
$gg \to gg$	LC	0.204–0.542	2.25†	$gg \to gg$	0.433–1.18	—	—	—
$gg \to gggg$	FC	139–409	3.61	$gg \to gggg$	474–1 410	985	3 800	2.69
$gg \to gggg$	LC	55.7–146	11.03	$gg \to gggg$	578–1 570	1 090	—	—
$gg \to ggggg$	FC	2 710–7 920	3.61	$gg \to gggg$	9 210–27 400	—	—	—
$gg \to ggggg$	LC	1 070–2 830	11.03	$gg \to gggg$	11 100–30 500	—	—	—
$u\bar{u} \to ggggg$	FC	827–2 630	3.30	$u\bar{u} \to gggg$	2 580–8 260	—	—	—
$u\bar{u} \to ggggg$	LC	206–526	12.66	$u\bar{u} \to gggg$	2 440–6 670	—	—	—
$u\bar{u} \to e^-e^+ggg$	FC	24.5–68.1	1.66	$u\bar{u} \to e^-e^+gg$	38.3–111	—	320	2.88
$u\bar{u} \to e^-e^+ggg$	LC	3.34–8.31	1.66†	$u\bar{u} \to e^-e^+gg$	5.21–13.9	—	—	—
$u\bar{u} \to e^-e^+d\bar{d}g$	FC	5.34–16.5	1.66	$u\bar{u} \to e^-e^+gg$	8.34–27.1	—	45.1	1.67
$u\bar{u} \to e^-e^+d\bar{d}g$	LC	1.19–3.14	1.66†	$u\bar{u} \to e^-e^+gg$	1.85–5.32	—	—	—
$gg \to t\bar{t}gg$	FC	54.8–156	2.68	$gg \to t\bar{t}gg$	138–408	190	633	1.55
$gg \to t\bar{t}gg$	LC	14.6–40.1	11.06	$gg \to t\bar{t}gg$	152–434	231	—	—
$gg \to t\bar{t}e^-e^+g$	FC	27.4–76.1	2.68	$gg \to t\bar{t}gg$	69.0–200	—	553	2.76

Design row	Col.	Recipe	Outer	Class	G1 estimate	Measured	Gate	Headroom
$gg \to t\bar{t}e^-e^+g$	LC	4.06–11.0	11.06	$gg \to t\bar{t}gg$	42.2–119	—	—	—

Table 8: **Schedule M against B** (native inputs, one thread), pure-gluon classes, the only classes where M merges seed entries (DESIGN §5.6). The schedule-dependent stages are seeds, factors, dressing, scatter and the NinjaRS reduction ("the reduction included", §1.6 item 5); the trees, Born and counterterm inputs and the glue are the same work in both schedules. M is adopted where M/B ≤ 0.85 on the schedule-dependent stages at both batch sizes; ± is the timing uncertainty of the ratio.

Process	Col.	Stages M/B, $B = 1$	Stages M/B, $B = 128$	End to end, $B = 1$	End to end, $B = 128$	Adopt M
$gg \to ggg$	FC	0.99 ± 0.00	1.00 ± 0.03	0.99	1.11	no
$gg \to ggg$	LC	0.90 ± 0.00	0.93 ± 0.05	0.93	0.92	no
$gg \to gggg$	FC	0.88 ± 0.01	0.85 ± 0.09	0.89	0.92	no
$gg \to gggg$	LC	0.75 ± 0.02	0.79 ± 0.01	0.86	0.94	yes

B against M (decision, provisional until the trees are compiled). On the schedule-dependent stages M passes the 15 % rule for $gg \to gggg$ at LC (as DESIGN §5.6 measured on the dressing side, 0.75–0.80), and not for the others: $gg \to gggg$ at FC lies at 0.85–0.88, the rule's edge within its uncertainty, and $gg \to ggg$ at 0.90–1.0. End to end the gain is diluted by the tree inputs, which both schedules share and which compiled trees will shrink (Table 4). With a quark line M's entries do not merge at all (M/B work 1.00, DESIGN §5.6; equal by test). **Decision (G1):** schedule M for pure-gluon classes at leading colour from $n = 6$; schedule B for every other class at $n \leq 6$, including pure-gluon full colour; M stays the planned default from $n \geq 8$, and both are re-measured at seven legs with the WP3.1 plans (`measure-g1-7leg`).

Table 9: **Values at the timed points**: pyAmpliCol's binary64 finite part (native inputs) against each provider's production-mode value on the common stream points (the providers read the binary64 decimal text of the exact points), largest deviation, metric $|a - b| \,/\, \max(|a|, |b|, |B|\ \alpha_s/2\pi)$; Recola's finite part converted with the universal double pole (it returns no poles in production). Worst point: the point of the row's largest deviation, evaluated again with one rotation and the reduction-direction check in binary64 and in double-double (DD): E is the binary64 estimate, DD the DD value's deviation from the closest provider (named). Campaign: the first three points against the committed references (`tests/fixtures/loop/required`), fixture metric and tolerance, native inputs and integrated.

Process	Pts	vs MadLoop	vs Recola	vs OpenLoops	Worst point: E	DD: closest provider	Campaign, native	Campaign, integr.
$gg \to gg$	16	1.7×10^{-12}	7.4×10^{-13}	7.5×10^{-13}	—	—	—	—
$u\bar{u} \to d\bar{d}g$	16	6.8×10^{-13}	3.3×10^{-10}	6.2×10^{-13}	pt 1: 7.2×10^{-12}	3.2×10^{-13} MadLoop	3.6×10^{-10}	3.6×10^{-10}
$u\bar{u} \to u\bar{u}g$	16	3.4×10^{-12}	7.2×10^{-11}	6.8×10^{-12}	pt 1: 9.2×10^{-12}	2.4×10^{-13} MadLoop	8.6×10^{-11}	8.6×10^{-11}
$u\bar{u} \to e^-e^+g$	16	8.8×10^{-14}	4.9×10^{-14}	no library	pt 0: 1.4×10^{-12}	7.5×10^{-15} Recola	—	—
$u\bar{u} \to ggg$	16	1.4×10^{-12}	1.5×10^{-12}	8.4×10^{-11}	pt 2: 1.2×10^{-12}	1.3×10^{-14} MadLoop	—	—
$gg \to ggg$	16	1.7×10^{-6}	1.7×10^{-6}	1.7×10^{-6}	pt 10: 9.9×10^{-6}	6.0×10^{-12} Recola	—	—
$u\bar{u} \to e^-e^+gg$	16	1.1×10^{-10}	5.0×10^{-11}	no library	pt 8: 2.5×10^{-9}	4.3×10^{-13} Recola	—	—
$gg \to gggg$	8	1.7×10^{-7}	4.0×10^{-7}	4.0×10^{-7}	pt 1: 4.8×10^{-6}	4.4×10^{-12} Recola	4.0×10^{-7} above 2.0×10^{-8}	8.7×10^{-8} above 2.0×10^{-8}
$gg \to t\bar{t}gg$	16	7.6×10^{-9}	1.5×10^{-9}	1.5×10^{-9}	pt 13: 5.6×10^{-8}	5.5×10^{-15} OpenLoops	1.1×10^{-11}	1.1×10^{-11}
$u\bar{u} \to gggg$	16	—	9.4×10^{-12}	1.9×10^{-10}	pt 9: 2.7×10^{-9}	1.9×10^{-13} OpenLoops	—	—

Reading the values. The Born agrees with every provider to 1.0×10^{-13} or better. For the finite part, the double-double (DD) value at each worst point (Table 9) separates the sources: it agrees with the closest provider to 7.3×10^{-12} or better everywhere. Where pyAmpliCol's binary64 estimate E exceeds the deviation — $gg \to ggg$, $gg \to gggg$ and $gg \to t\bar{t}gg$ (above 10^{-9}) and $u\bar{u} \to gggg$ (1.9×10^{-13} in DD against 1.9×10^{-10} in binary64) — the deviation is pyAmpliCol's own binary64 loss, which E flags. The other deviations above 10^{-11} stay in DD and are the providers' own accuracy: Recola's at $u\bar{u} \to d\bar{d}g$ (3.3×10^{-10}), $u\bar{u} \to u\bar{u}g$ and $u\bar{u} \to e^-e^+gg$ (COLLIER's default target 10^{-8}, HANDOFF §7), OpenLoops' at $u\bar{u} \to ggg$ (8.4×10^{-11}) and MadLoop's at $u\bar{u} \to e^-e^+gg$ (1.1×10^{-10}); MadLoop's production values also stay off the DD values at the ill-conditioned gluon and top points (up to 2.4×10^{-7}). At the campaign points $gg \to gggg$ point 1 misses the fixture tolerance (2×10^{-8}) in binary64 on both paths, within its estimate $E = 4.8\times10^{-6}$; its DD value agrees with Recola to 4.4×10^{-12}.

2 Required processes: references and generation

The plan requires the processes of Table 10 (plan §5.7), compared with MadLoop and Recola, and with OpenLoops where a public library exists. A reference provider is run only if its generation completes in about 45 minutes, or in a time comparable to pyAmpliCol's own generation of the process (HANDOFF §0.1, addendum F). Otherwise pyAmpliCol

is validated by Ward and gauge identities, Catani poles, LC/NLC/FC and DD/AP consistency, stability checks, lower multiplicities and existing references.

Table 10: **Generation on this host.** Providers: MadLoop graph generation, export and compilation; Recola process generation; OpenLoops library availability (HANDOFF §0.1). pyAmpliCol: v2 generation + plan export (`LoopSlice.generate`, then plan v4 JSON), plan size and peak resident memory, one thread; a development measurement on `work/v2-generation-scale`, not part of the acceptance campaign. Values stand side by side without a ratio.

Process	pyAmpliCol v2, dev.	MadLoop	Recola	OpenLoops	45-minute rule
$gg \to ggggg$	358 s + 109 s 185 MB · 5.8 GiB	> 17 h graph generation alone; colour factors ≈ 89 days (MG5 estimate)	> 57 h, unfinished	no public library	stopped 22 Oct; not to be restarted
$u\bar{u} \to ggggg$	193 s + 69 s 70 MB · 2.4 GiB	12.2 h (5 793 + 38 040 + 122 s)	2.8 h (10 220 s)	no public library	over 45 min; results exist and are used
$gg \to gggg$	26.7 s + 5.8 s 15 MB · 0.55 GiB	2.4 h (285 + 8 354 + 21 s)	34 min (2 036 s)	prebuilt ppjjjj	MadLoop over 45 min; results exist and are used
$gg \to t\bar{t}e^-e^+g$ EW	not generated	11 min	22 s	ppllttj_ew	within 45 min
$gg \to t\bar{t}e^-e^+g$ QCD	10.1 s + 3.6 s 2 MB · 0.2 GiB	2.7 min	29 s	ppllttj	within 45 min
$gg \to t\bar{t}gg$	20.9 s + 2.9 s 5 MB · 0.29 GiB	3.6 min	63 s	pptttj	within 45 min
$u\bar{u} \to e^-e^+ggg$	9.9 s + 3.7 s 2.3 MB · 0.22 GiB	2.4 min	16 s	pplljjj	within 45 min
$u\bar{u} \to e^-e^+d\bar{d}g$	9.5 s + 3.4 s 1.1 MB · 0.2 GiB	56 s	8 s	pplljjj	within 45 min
$2 \to 2$, $2 \to 3$ controls	—	< 1 min	< 2 s	—	within 45 min

Table 11: **Validation of the required set** (development , `work/v2-generation-scale`; binary64, all three Laurent coefficients, three exact points per process, trees, Born and counterterms from the Python per-point inputs). Deviation metric $|a-b| \,/\, \max(|a|,|b|,|B|\ \alpha_s/2\pi)$. Points that the stability estimate flags in binary64 are resolved in DoubleFloat. ML: MadLoop; OL: OpenLoops.

Process	References	Agreement
$gg \to gggg$	Recola, OL	Recola, OpenLoops, all coefficients: ≤ 8.9×10^{-12} at point 0 and ≤ 4.4×10^{-11} at point 2 (largest: finite part); point 1: 8.4×10^{-9} in binary64 (flagged, estimate 3.0×10^{-6}), 7.1×10^{-11} in DoubleFloat
$gg \to t\bar{t}gg$	ML, Recola, OL	MadLoop, OpenLoops ≤ 2.1×10^{-11}; Recola ≤ 1.0×10^{-10}
$u\bar{u} \to e^-e^+ggg$	ML, Recola, OL	all three providers ≤ 6.9×10^{-13}
$u\bar{u} \to e^-e^+d\bar{d}g$	ML, Recola, OL	all three providers ≤ 8.5×10^{-13}
$gg \to t\bar{t}e^-e^+g$ QCD	ML, Recola, OL	≤ 3.6×10^{-12} at two points; third point 5.7×10^{-9} in binary64 (estimate 1.7×10^{-9}), MadLoop to 1.9×10^{-14} in DoubleFloat
$gg \to t\bar{t}e^-e^+g$ EW	ML, Recola, OL	not started: Feynman-gauge massive vectors, Goldstones, complex-mass scheme, generic EW counterterms (WP3.2)
$u\bar{u} \to ggggg$	ML, Recola	MadLoop ≤ 2.6×10^{-12}, Recola ≤ 6.1×10^{-10} (finite ≤ 2.9×10^{-10}), all coefficients, three points
$gg \to ggggg$	NJet (no gate)	Born and poles against NJet (§3); finite part waits for the native counterterms

Table 12: **Provider references for the seven-leg gates**, s per point, one pinned core, production mode (finite part; OpenLoops `truepoles 0` with I-operator poles, Recola without IR poles, MadLoop production build), measured on 2026-10-20 for the design (DESIGN §1.3, Table 1b); inferred values of that table are omitted. G1 re-measures them with the seven-leg rows (`measure-g1-7leg`); the $2 \to 3$ and $2 \to 4$ references are in Table 1. Threshold: the largest pyAmpliCol finite-only time that would pass, 1.3× the fastest matched Recola/OpenLoops time where it is below MadLoop's.

Process	OpenLoops	Recola	MadLoop	Threshold
$u\bar{u} \to e^-e^+ggg$	0.348–0.353	0.246–0.259	1.41	0.32
$u\bar{u} \to e^-e^+d\bar{d}g$	0.053–0.055	0.035	0.143	0.045
$gg \to t\bar{t}e^-e^+g$ QCD	0.425–0.433	0.59–0.67	3.03–3.04	0.55
$gg \to t\bar{t}e^-e^+g$ EW	0.96–1.12	0.95–1.02	6.90–7.09	1.24
$u\bar{u} \to ggggg$	no library	not measured	not measured	—
$gg \to ggggg$	no library	none (§0.1)	none (§0.1)	no gate

3 Seven gluons: NJet and pyAmpliCol

3.1 NJet: Table 5 of arXiv:1209.0100, re-measured on this host

NJet is the independent reference for $gg \to ggggg$, where MadLoop and Recola cannot generate under the 45-minute rule and OpenLoops has no library. It is massless five-flavour QCD, and it is not one of the gate's matched providers.

Table 13: **NJet, s per point**, double precision, two evaluations each (NJet's scaling test). Paper: Xeon E3-1240 at 3.3 GHz, NJet 1.0. This host: one pinned core, idle at launch, of the shared host; 10 frozen points per channel, mean (spread ≤ 2 %); the multiplier is time ÷ the paper's. De-symmetrised sums equal the standard sums only after phase-space integration; they are timing figures, not point values.

Channel	Table 5	NJet 3.1.2		NJet 2.1.1		Table 5	NJet 3.1.2		NJet 2.1.1	
	standard full-colour sum					de-symmetrised sum				
$gg \to gg$	0.03	0.00295	×0.1	0.00291	×0.1	0.03	no class		no class	
$gg \to ggg$	0.22	defective channel		0.141	×0.6	0.07	defective channel		0.0452	×0.6
$gg \to gggg$	6.19	4.59	×0.7	4.58	×0.7	0.57	0.39	×0.7	0.38	×0.7
$gg \to ggggg$	171.31	142	×0.8	246.7	×1.4	3.07	2.41	×0.8	4.19	×1.4

At seven gluons one NJet 3.1.2 evaluation without the scaling test takes 71.2 s (standard) and 1.22 s (de-symmetrised), at 69 MB; double-double costs 1 990 s per point. NJet 3.1.2's numerical five-gluon channel is 9/2 too large (NJet 2.1.1 is used there), and at the unrotated seven-gluon points NJet keeps only 4–6 digits in double precision (NaN at 3 of 10), so the reference values are taken at exactly rotated points.

3.2 pyAmpliCol development measurements

These are development measurements on the branches named, on the shared host. Generation and the counterterm program ran in one thread; the bare-loop job's thread count was not recorded, so its wall times are not per-core timings (Table 14). They are not part of the acceptance campaign and not a like-for-like speed comparison: the native path below does not yet include counterterms or R_2.

Table 14: **$gg \to ggggg$ in full colour, development runs.** Wall times and peak resident memory from `/usr/bin/time` for the whole process, including model loading. Generation and the counterterm program are single-threaded (CPU share 101 %, 101 % and 99 %; `RAYON_NUM_THREADS=1` for the counterterm program). The bare-loop job did not record its thread count (the native executor's default is all cores); the whole job, generation and export included, averaged 168 % CPU. Its 1 769 CPU-s (user + system), less one core over generation (471 s) and export with trees (418 s), leave at most 879 CPU-s for the 146 s of evaluations: up to 6.0 busy cores on average and up to 176 CPU-s per point, so the per-core comparison with NJet is undetermined. Counts of the generated slice: 9 414 families, 810 495 skeletons, 197 539 trie nodes, 4 718 tree blocks, 1 080 colour-seed classes on a 720-element Born basis, 4 837 counterterm sites.

Run (branch)	Stages	Wall	Peak memory
Generation and plan, first run `work/v2-generation-scale`, 03:55 UTC	`LoopSlice.generate` 7.4 min (peak 4.2 GiB), plan export 5.4 min, JSON 38.7 s; 949 MB plan	13.6 min	9.3 GiB
Generation and plan, latest run same branch, 10:42 UTC	`LoopSlice.generate` 6.0 min (peak 4.2 GiB), plan export 1.8 min, JSON 9.05 s; 185 MB plan (derived row links)	8.0 min	5.8 GiB
Counterterms in the recursion `work/v2-ct-recursion`	Plain blocks 9.4 min, CT program 53.5 min; 3 070 042 CT currents, 34 781 532 interactions, 847 root sites, no rule left to site closure; script total 63.4 min. Root-closure (bind) terms not measured; run before the request-scoped lowering (ct-recursion DESIGN §6.1)	66.4 min	52.1 GiB
Bare loop + Born, native trees `work/v2-generation-scale`, 5 points	Binary64; per point the point and one rotation (two evaluations); no counterterms, no R_2; thread count not recorded (up to 6.0 cores, ≤ 176 CPU-s per point)	25.5–41.3 s/pt	18.5 GiB whole job

On the same host (Table 13) NJet takes 142 s per point for its standard sum and 2.41 s for its de-symmetrised sum, two evaluations each, on one pinned core, idle at launch, of the shared host. The native path's 25.5–41.3 s per point (also two evaluations) are wall times of a multi-threaded job: its CPU accounting allows up to ≈6.0 busy cores, ≈176 CPU-s per point, during the evaluations, which is above NJet's 142 s. Whether the native bare path is faster than NJet's standard sum per core is therefore undetermined until a one-thread run exists. The comparison is not like-for-like either: counterterms and R_2 are missing on the pyAmpliCol side, its job ran unpinned and multi-threaded on the same shared host, and the de-symmetrised sum is not a point value. Generation fits addendum H's budget (O(hours), below 150 GB). MASTER_PLAN quotes the same two peaks as KiB/10^6 ("9.7 GB", "54.6 GB"); of the run records, the counterterm record prints KiB/10^6 and the generation record GiB.

3.3 Validation without a matched provider

Table 15: **Checks of the seven-leg QCD results**, development measurements on `work/v2-generation-scale`, not part of the acceptance campaign, except the Ward row (WP2.5 on `loop`); binary64, deviation metric of Table 11. NJet values: fixture `gg_ggggg_njet.json` (NJet 3.1.2 double-double at exactly rotated points, scaling-test estimates ≤ 1.3×10^{-24}). The bare single pole is compared after subtracting the counterterm pole of its request, as marked in each row: $(\alpha_s/4\pi)B[(n-2)\beta_0^{(5)} + 8T_F/3] = (\alpha_s/4\pi)B \cdot 119/3$ for loop_sm (five-flavour coupling renormalisation and the top's decoupling term) and $(\alpha_s/4\pi)(n-2)\beta_0^{(5)}B = (\alpha_s/4\pi)B \cdot 115/3$ for the top-free request.

Check	Result	Status
$gg \to ggggg$, native bare loop, 5 points: double pole ÷ Born against $-(\alpha_s/2\pi)\cdot 7C_A$	≤ 5.4×10⁻¹²; rotation estimates 1.6×10⁻¹⁰–2.9×10⁻⁶	passed
$gg \to ggggg$ against NJet, native bare loop (loop_sm, 5 points; shift 119/3)	Born ≤ 1×10⁻¹³, double pole ≤ 5.4×10⁻¹², single pole 3.5×10⁻¹²–8.3×10⁻¹⁰ at points 0–3; 7.2×10⁻⁸ at point 4, which pyAmpliCol's estimate (2.9×10⁻⁶) flags	no deviation beyond the estimates
$gg \to ggggg$ top-free (NJet's scheme) against NJet, native bare loop, 3 points; shift 115/3	Born ≤ 4.9×10⁻¹⁴, double pole ≤ 5.4×10⁻¹², single pole 3.4×10⁻⁹ and 5.2×10⁻⁹ (estimates 1.4×10⁻⁸, 5.0×10⁻⁸), 1.4×10⁻¹¹	finite part needs the native counterterms
Top-free full virtual against NJet at lower multiplicity	$gg \to gg$: ≤ 2.1×10⁻¹⁴ at 10 points, full virtual; $gg \to ggg$: ≤ 2.3×10⁻¹³ at the stable points; the flagged point 3×10⁻¹⁴ in DoubleFloat; $gg \to gggg$: 1.0×10⁻¹¹ at point 0; point 1 1.6×10⁻⁷ in binary64 (estimate 2.7×10⁻⁶), 4.7×10⁻¹⁵ in DoubleFloat (`test_loop_v2_topfree.py`)	passed
$u\bar{u} \to ggggg$ full virtual against MadLoop and Recola, 3 points	MadLoop ≤ 2.6×10⁻¹², Recola ≤ 6.1×10⁻¹⁰ (finite ≤ 2.9×10⁻¹⁰); the providers agree to 6.2×10⁻¹⁰	passed
Ward identity, one gluon's $\varepsilon \to p$, at seven gluons	asymmetric Ward mode of WP2.5 (loop side only, bare + UV + R_2); passes on $gg \to gg$ and $u\bar{u} \to ggg$ (`test_loop_v2_observables.py`)	not yet run at seven gluons (addendum H)

First $gg \to ggggg$ virtual development `work/v2-generation-scale`, not part of the acceptance campaign (full colour, Born α_s^5, loop_sm with top loops, point 0, $\mu = 1$ TeV; Python per-point inputs with counterterms, about 3 h per evaluation on that development path):

	Born	finite	$1/\varepsilon$	$1/\varepsilon^2$
value, GeV^{-6}	3.62326×10⁻⁶	−1.30223×10⁻⁵	−6.93874×10⁻⁶	−1.42896×10⁻⁶
rotated copy, rel. diff.		5.9×10⁻⁹	2.6×10⁻⁹	2.5×10⁻¹³
NJet (no top), deviation	3.8×10⁻¹⁴	top loops	1.2×10⁻⁹	1.1×10⁻¹²

The finite part differs from NJet's by the top-loop contribution (−1.34×10⁻⁸, i.e. −0.197 $|B|\ \alpha_s/2\pi$), which continues the sign and size seen at four to six legs; it is not a test.

4 Capabilities at this milestone

Table 16: **Capability matrix** of the v2 engine on `loop` at b4d76cc3, with branch work marked.

Feature	Status	Boundary
Recursive engine	supported	Trees, families and loop numerators from recursions; no loop diagram is enumerated (FeynKit graphs are test oracles only). Counterterms: see below.
Full, NLC and LC colour	supported	Exact seed tables in N_c; LC/NLC by generated classes and truncated cells. LC is cheaper in the loop stage but not yet end to end at large batches (its native trees cost more, Table 5), nor to generate at seven legs.
Colour and spin correlations	supported	Directed colour histories at LC/NLC/FC, spin sources, resolved helicities, flavour aliases, Ward sources (WP2.5). Seven-leg directed LC/NLC tables need the Rust seed tables (WP3.1).
Per-order partials	supported	Every squared coupling order at tree level and at one loop, mixed Born orders, per-correction loop orders.
Poles at runtime	supported	`poles = "all" | "none" | "uv" | "ir" | "split"` (WP2.10, on `loop` since 24 October): finite-only mode with the finite part bitwise unchanged and the pole work skipped; `"uv"` and `"ir"` with the user's semantics of 23 October. The UV residue comes from an interim local module until the NinjaRS re-pin; a `"uv"` estimate needs a rotation check.
Stability estimates	supported	Rotation checks (any number, 0 = estimate absent), reduction direction, exact rescaling, in any combination; no automatic escalation (WP2.3).
Precision carriers	supported	Binary64, DoubleFloat, arbitrary precision and exact rationals in the native executor; DD/AP point inputs are formed in Python Decimal until WP3.1.
Native executor, schedules	supported	Rust, 16 lanes (also for a one-point call, a G1 lever), threads; G1 decision on the schedule-dependent stages (Table 8, provisional until compiled trees): M for pure-gluon LC from $n = 6$, B for every other class at $n \le 6$; F as oracle. The NinjaRS reduction runs serially. Development report: stage times, modelled kernel work, master-integral time.
Plan v4 artifacts and API	partial	Published and loaded through Python, TOML and CLI; relocation to another path tested on the same host and CPU. Loading regenerates the process (the `LoopSlice`) in Python and checks the plan digest, which addendum G forbids for prepared artifacts; WP3.1 removes it. The Rust/C/C++/Fortran SDK refuses v4 until WP3.1.
Counterterms (model UV, R_2)	supported	Plain tree vertices of the recursion inserted exactly once (WP2.2, a18b7599; addendum J, b4d76cc3), admitted by coupling orders and the public `particle_veto`; no witness table or loop matching in production. An unlowerable counterterm vertex that no tree of the request reaches is inert and recorded in the artifact; a reachable one is refused. Site closure is the test oracle. Per point they are formed in Python on `loop`, natively on the WP3.1 branch.
Direct R_2	deferred	A bonus by the user's rule; bounded controls only.

Feature	Status	Boundary
Electroweak, complex mass	open	Tree EW lanes fixed, the Dirac–Weyl class refused; loop EW, Goldstones and CMS are WP3.2.
Required set	in progress	On the WP3.1 branch seven of eight generate and six agree with their provider references in all coefficients (Table 11); $gg \to ggggg$ is validated in Born and poles; the EW target is not started.
Performance gates	in progress	G1 measured (Section 1): the integrated slice fails every $2 \to 3$ and $2 \to 4$ gate it generates (Python per-point inputs) and does not generate V+jets (WP3.5, WP3.2). With the WP3.1 native inputs at $B = 128$: 7 of 9 pass, $u\bar{u} \to u\bar{u}g$ open, $u\bar{u} \to d\bar{d}g$ fails; $B = 1$ needs lane adaptation (Table 2). Missing: provider protocol-mode columns, LC for V+jets, seven legs (`measure-g1-7leg`).
$gg \to 6g$	deferred	No $2 \to 6$ work until the $2 \to 5$ plan is complete (user, 23 October).

5 Architecture of the v2 engine

5.1 The computed quantity

For Born order a, loop order b and squared coupling order $s = a + b$,

$$V_{s,k} = \mathcal{N} \sum_{a+b=s} 2\,\mathrm{Re}\big[B_a^\dagger G_{BL} L_{b,k}\big], \quad k \in \{0,-1,-2\}, \quad V = C_\varepsilon \left(V_0 + \frac{V_{-1}}{\varepsilon} + \frac{V_{-2}}{\varepsilon^2}\right), \quad C_\varepsilon = \frac{(4\pi)^\varepsilon}{\Gamma(1-\varepsilon)},$$

where G_{BL} is the colour metric between Born and loop colour structures and $\mathcal{N}$ holds the initial-state averages and identical-particle factors (MadLoop/BLHA convention; couplings included, nothing divided by the Born). Directed colour and spin correlations replace $2\,\mathrm{Re}(\ldots)$ by the two cross terms $B_A^\dagger G L_B + L_A^\dagger G B_B$.

5.2 Pipeline

For every denominator family f, sector s and observable column o, v2 builds the four-dimensional numerator $N_{f,s,o}(q) = \sum_\alpha c_\alpha q^\alpha$ that NinjaRS reduces: Born-weighted and helicity-summed, produced by three recursions that share all work (Figure 1).

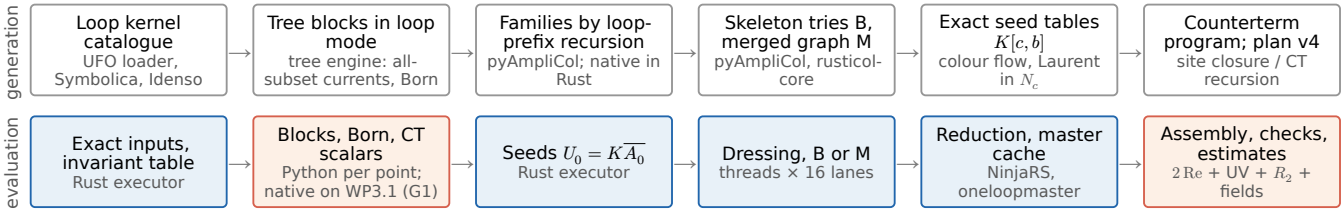


Figure 1: **v2 pipeline.** Upper lane: generation, once per process, into a plan v4 artifact. Lower lane: evaluation per batch of points. Blue: native batched numerics in the Rust executor; orange: still formed in Python per point (native lanes on the WP3.1 branch); white: generation. The owning code or dependency is named under each step.

Trees. The tree engine's colour-ordered Berends–Giele recursion computes, once per point, every off-shell current over every leg subset a loop vertex can absorb (the *blocks*), the Born colour components A_0 and, with WP2.2, every tree with one counterterm vertex. A loop-aware reachability test keeps exactly the currents some family position or root needs. Blocks with transverse sources use the exact transverse Yang–Mills kernels; loop segments, counterterm currents and Ward sources use the full ones.

Families and skeletons. A Recola-type recursion over loop prefixes (ordered leg sets, propagator species and masses) yields the denominator families, keyed by the canonical cyclic sequence of (leg mask, mass) and equal to Recola's tensor-integral identities. Per family it yields the colour-ordered *skeletons*: rings of blocks, each block summing all trees on its leg subset (Figure 2). Skeletons are never stored as a list; they are the leaves of the schedule tries. Only certified structural zeros are dropped (scaleless rings, external wave-function corrections, VEV tadpoles, colour zeros); nothing is restricted to 1PI.

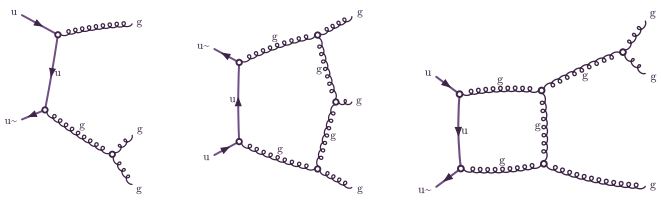


Figure 2: Three graphs of $u\bar{u} \to ggg$ drawn by FeynKit's renderer from its own generation (an oracle, not a production input). In v2 the Born (left) is a closure of blocks; the pentagon (middle) is a ring of five single-leg blocks; the box with a tree attachment (right) is a ring of four blocks, one of them the two-leg current, computed once per point and shared by every skeleton that contains that leg subset.

Loop numerators and schedules. An open-loop recursion dresses skeletons one segment at a time with polynomial-valued objects, $O' = O \cdot Y + \mathrm{raise}\big(O \cdot Z_\mu, q^\mu\big)$, in the NinjaRS monomial layout ($\binom{r+4}{4}$ coefficients at rank r, no rank cap). The seed is the colour-summed Born interference, and each block's helicities are summed as it is attached. One generator feeds three schedules (Table 17).

Table 17: **Three schedules on one generator** (DESIGN §2.2, §5.6).

	F: forward	B: suffix trie (default)	M: merged, primitive-seeded
Born weighting	at closure	at the leaves (one seed per skeleton class)	at the cut, one seed per colour ordering
Shared	prefixes across families	all prefixes with equal family and remainder	prefixes across families and compositions
Colour certificate	none needed	none needed	certified classes: adjoint rings, traces, quark rings
Role	oracle	default; generic fallback	pure-gluon LC from $n = 6$ (G1); planned default from $n \geq 8$

Colour. Colour lives only in exact seed tables $K[c, b]$, Laurent polynomials in N_c between seed classes c (the colour vector obtained by closing the cut of a skeleton's colour flow) and Born basis elements b; per point the seeds are $U_0 = \sum_b K[c, b]\overline{A_0[b]}$, with a symmetric-group FFT where certified. LC and NLC differ from full colour only in which classes are generated and how cells are truncated: at LC each colour ordering is seeded by its own Born amplitude and skeletons below the leading N_c degree are never generated. Certificates (adjoint rings, fundamental traces, quark rings) are proved at loop level, never inherited from the tree level.

Reduction and masters. Families close directly into NinjaRS `TensorNumerator` lanes: one reduction per (family, sector, column), families with equal (propagators, rank) batched as lanes of one call, scalar masters from oneloopmaster cached per event. A second reduction in a relabelled frame reuses the numerators and the cached masters as the cheap stability check.

Counterterms. The model's UV and R_2 vertices, two-point ones included, are ordinary vertices of the tree recursion, and each contribution contains exactly one of them; external field factors are separate sources. Admission is the requested coupling orders, the exactly-once constraint and a public `particle_veto`: a coupling term whose loop particles include a vetoed particle is dropped, and a vetoed particle is removed from every tree vertex, loop line and propagator of the generation, so loops and counterterms stay consistent. Nothing is matched to the loop (user, 23 October; addendum J). On `loop` the counterterms come from the WP2.2 recursion (one counterterm order in the tree recursion, the default route, whose $gg \to ggggg$ program leaves no rule to site closure, Table 14) with this admission (`ct-simplify`, b4d76cc3): no witness table and no loop matching. A counterterm vertex that the model cannot lower and that no tree of the request reaches (the tree engine's own reachability, with the program's order limits and `particle_veto`) is inert and recorded in the artifact; a reachable one is refused. `particle_veto = [t]` replaces the internal top-free switch of Table 15; site closure and the witness machinery remain only as test oracles.

Precision, stability and assembly. Every kernel is generic over binary64, DoubleFloat, arbitrary-precision floats and exact rationals. A failed lane fails its point and never branches a kernel. The assembly takes $2\,\mathrm{Re}$ of $i/(16\pi^2)$ times the reduced family sum, plus local UV, R_2 and field terms, per sector and column, with optional rotation, reduction-direction and rescaling estimates and the finite-only mode. The public API selects `loop_engine = "v2"` from TOML, CLI or Python and stores plan v4 as a relocatable PACBIN member (zlib JSON of exact kernel term lists, no compiled code, digest checked at load).

Table 18: **Asymptotic classes per event** for n gluons, full colour (DESIGN §1.4): $F_n \approx (n-1)!\big(\frac{1}{\ln}2\big)^n/2$ families, 2^n helicities. The required set ($n \leq 7$) is far below the crossover of schedule B with the unitarity class ($n \approx 30$–50); schedule M is planned as the default from $n \geq 8$.

Stage or code	Class
Tree blocks	$\Theta(n \cdot n!)$ currents; $\Theta(n! \cdot 2^n)$ current-helicity rows
Dressing, schedule B	$\Theta((n-1)! \cdot 4^n)$ seed injection; trie dressing dominates for $n \leq 8$
Dressing, schedule M	$\Theta(n! \cdot 2^n \cdot n^4)$
NinjaRS	$\Theta(F_n)$ reductions, $\Theta(F_n \cdot n^8)$ work, $\Theta(4^n)$ distinct masters after caching
MadLoop (per diagram, per helicity)	$\Theta(D_n \cdot 2^n \cdot n^4)$, D_n super-exponential ($gg \to gggg$: 26 975 loop diagrams)
Unitarity with exact helicity sums (NJet)	$\Theta((n-1)! \cdot 2^n \cdot n^{6\ldots9})$

Table 19: **Ownership of the mathematics.** pyAmpliCol reuses its dependencies for algebra, tensors, graphs and integrals and owns the recursions and the assembly.

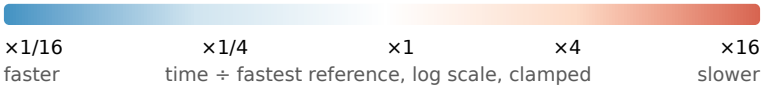
Component	Responsibility
ufo_model_loader	Lossless UFO loop-model import: Laurent-valued parameters and couplings, UV/R_2 categories, loop groups
Symbolica	Expressions, coefficient extraction in q, evaluators; exact rationals and arbitrary precision
Spenso, Idenso (Community)	Lorentz and colour tensors behind the kernel catalogue and the colour metrics
FeynKit (Community)	Test oracles: graph and family counts, overall factors, drawings (Figure 2)
NinjaRS, oneloopmaster	Integrand reduction with Laurent expansion; scalar masters in all carriers
pyAmpliCol tree engine	Blocks, Born, coupling orders and colour flow (Python lane; Rust recurrence)
pyAmpliCol loop v2	Families, skeletons, schedules, seed tables, counterterm program, plan v4, native executor, API

6 History: legacy per-diagram engine, 2026-10-18

History only. These timings measure the superseded per-diagram engine (per-graph coefficient programs, pairwise colour overlaps, whole-process symbolic projectors) through its Python coordinator with exact decimal input, on repeated three-point streams, against providers that kept their protocol settings (actual poles, stability checks and rescue). They are neither the gate protocol nor the v2 engine, include $2 \to 2$ processes, and are kept for reference without gate counts. Revisions fa599885 and two follow-ups; AMD EPYC 9754 128-Core Processor.

Table 20: **Legacy engine, µs per point** (2026-10-18), binary64, full colour, all three Laurent coefficients; median of five warm calls; multiplier: time ÷ the fastest timed reference of that row, coloured only where MadLoop is timed. Grey rows: generation (pyAmpliCol fresh process to plan; MadLoop export and compilation; Recola generation; OpenLoops setup of a prebuilt library). B: points per call.

Process	B	pyAmpliCol (legacy)		MadLoop		Recola		OpenLoops	
$u\bar{u} \to e^-e^+$ photon only · 1 loop graphs	1	584	×12.5	46.9	fastest	1 880	×40.1	no matched control	
	128	147	×3.2	46.5	fastest	1 780	×38.4	no matched control	
	gen.	2.43 s		198 s		0.135 s		—	
$u\bar{u} \to d\bar{d}$ QCD · 14 loop graphs	1	1 730	×6.9	251	fastest	2 650	×10.5	548	×2.2
	128	764	×3.0	254	fastest	2 670	×10.5	563	×2.2
	gen.	7.09 s		14.4 s		0.253 s		setup 0.0382 s	
$u\bar{u} \to t\bar{t}$ QCD · 14 loop graphs	1	2 010	×5.7	352	fastest	2 910	×8.3	655	×1.9
	128	745	×2.0	376	fastest	3 050	×8.1	686	×1.8
	gen.	6.39 s		17.1 s		1.12 s		setup 0.0103 s	
$u\bar{u} \to gg$ QCD · 45 loop graphs	1	6 830	×14.8	463	fastest	5 340	×11.5	1 700	×3.7
	128	3 100	×8.3	372	fastest	4 470	×12.0	1 700	×4.6
	gen.	21.2 s		29.1 s		0.183 s		setup 0.0149 s	
$gg \to gg$ QCD · 186 loop graphs	1	33 700	×13.0	2 590	fastest	36 100	×13.9	11 900	×4.6
	128	14 300	×6.9	2 070	fastest	33 200	×16.0	16 200	×7.8
	gen.	98.7 s		30.1 s		0.170 s		setup 0.0130 s	
$u\bar{u} \to ggg$ QCD · 556 loop graphs	1	194 000	×5.4	36 100	fastest	81 500	×2.3	36 900	×1.0
	128	125 000	×3.7	33 500	fastest	88 600	×2.6	43 400	×1.3
	gen.	863 s		23.9 s		0.724 s		setup 0.0109 s	
$e^-e^+ \to \mu^-\mu^+$ full EW, G_μ · 98 loop graphs	1	6 590	×2.1	validated, not timed		3 100	fastest	not timed	
	128	2 920	×1.0	validated, not timed		2 950	fastest	not timed	
	gen.	116 s		—		0.175 s		—	



The per-diagram design could not reach the $2 \to 5$ set: a seven-leg generation ran for more than nine hours without a result, and its counterterm enumeration alone needed up to 77 GiB. The v2 engine replaced its numerator, colour and counterterm core (Section 5) and keeps model import, the counterterm rules, Born weighting before reduction, the NinjaRS/oneloopmaster adapter, the exact carriers and the public interfaces.

7 Sources and reproducibility

- **Build:** `typst compile benchmarks.typ benchmarks.pdf` in `docs/performance_reports/loop/` (Typst 0.15).
- **Status, in-flight work, decisions:** `data/milestone.json`, from the `Integrate` … merges of b4d76cc3 since f132e53c `workspace/reports/engine-v2/slice-status.md` (integration blocks, DECISIONS 2026-10-23/24); `HANDOFF_24_10_2026.md`; MASTER_PLAN 2026-10-23/24 entries; REQUIREMENTS_ADDENDUM A, D, F, G, H, I, J; DESIGN §5.7 (G1), §6.1, §6.2, §12.4; the 24 October pushes (59badc7a and b4d76cc3, gates in slice-status.md).

- **Table 10:** provider columns HANDOFF §0.1 (addendum F); pyAmpliCol column DESIGN WP3.1 third increment "Generation per process" on `work/v2-generation-scale` (201efa8b). **Table 11:** DESIGN WP3.1 first to fourth increment, same branch (records `workspace/tmp/generation-scale/runs/`); the *gg* → *gggg* row from `cmp_gg_gggg.json`, `cmp_gg_gggg_p1_stab.json` and `cmp_gg_gggg_p1_dd.json` there. **Table 12:** DESIGN §1.3 Table 1b. **Conventions:** MadLoop standalone, Recola, OpenLoops 2.
- **Section 1 (Table 1 to Table 9):** `data/g1.json` and the per-point values `data/g1-values.json`, written by `python -m tools.loop_benchmark gate` (README and docstrings of `tools/loop_benchmark`); streams `tools/loop_benchmark/streams/`; run records archived in `workspace/reports/engine-v2/measure-g1/records/` (first campaign `runs/`, fix round `runs2/`), MadLoop exports of the G1-only processes in `workspace/reference/runs/g1/`; recalibration `tools/loop_v2_costmodel.py --g1 data/g1.json`. Builds: `loop-native-measure-g1-033edf15` (integrated, this branch at `1c971289`) and `loop-native-measure-g1-genscale-39824f9b` (`work/v2-generation-scale` 889af02c plus the timers; scratch commit `39824f9b`, kept as tag `g1-native-inputs-39824f9b`), release with one codegen unit and thin LTO; the native-input harness is this branch's `tools/loop_benchmark`. Schedule M at seven legs: DESIGN §5.6.
- **Table 13** and NJet prose: `workspace/reports/engine-v2/njet-benchmark.md` §2, §5, §6 (raw `workspace/reference/runs/njet/timing/`); NJet, arXiv:1209.0100 Table 5.
- **Table 14, Table 15, first virtual:** `data/measured.json` (`scripts/collect_milestone.py <workspace>` from the records in `workspace/tmp/generation-scale/runs/` and `workspace/tmp/ct-recursion/s4/`); NJet deviations: njet-benchmark.md §5.3 and `gg_ggggg_njet.json`; top-free values: DESIGN WP3.1 and `test_loop_v2_topfree.py`. Memory in GiB = KiB/2^{20}.
- **Table 16, Section 5:** `docs/development/loop_v2/DESIGN.md` §1–§3, §5.5, §5.6, §6, §10, §12, REQUIREMENTS_ADDENDUM J (counterterm rule) and the integration records above.
- **Table 20:** `data/measurements.json`, `data/provider-cold.json` and `data/legacy-labels.json` (legacy snapshot of 2026-10-18, unchanged); `data/colour-measurements.json` stays for the DESIGN citations; removed data are in the git history (README).